

RANDOM ORACLE MODEL :-

HASH FUNCTIONS are fundamental cryptographic primitives that must satisfy a few important properties:

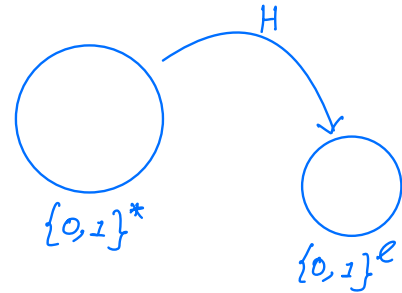
- Deterministic
- Fixed output size
- Avalanche effect (Small changes in the input produce significantly different outputs)
- One way computation
- Collision resistance

Above all, collision resistance is particularly important and is measured by the difficulty of finding two distinct inputs that hash the same value.

But many cryptographic constructions require properties from hash functions that go beyond collision resistance.

Eg. In key derivation functions, commitment schemes and certain other signature schemes, we need hash functions to behave in ways that are difficult to formalize using standard cryptographic assumptions. Unfortunately NO SECURITY PROOFS AVAILABLE for such primitives based on standard properties of hash function.

Referring to collision resistance property.



What exactly RANDOM ORACLE MODEL does is, it provides a way to prove security for schemes (which are designed based on hash functions) where we cannot give the security proofs based solely on the standard properties like collision resistance.

WHAT IS RANDOM ORACLE MODEL?

The Random oracle model assumes the existence of a truly random function that is accessible to all parties through oracle queries. Not knowing the internal code/logic

Key assumptions:

1) Each participant (including adversaries) has oracle access to the hash function H .

i.e., participants can query $H(x)$ and receive y , but cannot see the internal implementation of H .

Let $H: \{0,1\}^* \rightarrow \{0,1\}^l$ be a function from arbitrary length binary strings to l -bit outputs.

2) If an input x have never been queried before, the oracle assigns a uniform random output $y \in \{0,1\}^l$ and stores (x,y) in its internal table.

Let $Q \subset \{0,1\}^*$ be the set of previously queried inputs.

If $x \notin Q$, then $H(x) \leftarrow \{0,1\}^l$, sampled uniformly at random

3) Once the oracle has generated $H(x)=y$ for some input x , it will always return the same y $\forall$ subsequent query $H(x)$

4) Each party's queries to the oracle are not observable by other parties (which prevents adversaries from learning about other's input/output)

5) In a reduction-based proof, the simulator A' that runs the adversary A can see all the inputs x_i that A query to the oracle

$\rightarrow$ EXTRACTABILITY

NOTE: But this looks like a contradiction to the earlier point?!

No contradiction really! Private channels protect against external attacker in the real world. But Extractability applies only during simulation in security proofs.

6) When writing a reduction based security proof, the simulator A' that models the system can decide in advance what the oracle returns for some specific input x , even before the adversary queries it. $\rightarrow$ PROGRAMABILITY

Formally,

A' can set $H(x) := y$ for any chosen x, y (as long as y is valid, uniform random value)

But why is this needed?

In proofs, we often want to reduce the security of a cryptographic scheme to a hard problem. (Eg DL, RSA, CDH, ...)

We want to show: "If an adversary A can break the scheme, then we can use A to solve a known hard problem"

To do this, A' might be given a challenge instance (G, g^a) and must embed this into the simulated protocol that A interacts with.

Programmability lets A' plant this challenge into the oracle's input (which is crucial for the reduction to work)

USE CASE: A canonical example of programmability is in the Fiat-Shamir signature scheme, where

- The prover commits to t
- Challenging, $c = H(t)$
- & the response is computed based on t and c .

In a security proof, the simulation is given a challenge c^* and it needs to program the oracle so that $H(t^*) = c^*$

But in real world, we cannot forge $\text{SHA-256}(t^*) = c^*$ unless we find a preimage.

But in ROM it's possible!

7) If protocol Π is run multiple times, each run uses a separate function $H_1, H_2, \dots, H_n$ all drawn independently from the uniform function space $\text{Func}(\{0,1\}^* \rightarrow \{0,1\}^l)$

prevents reuse of oracle randomness between sessions

8) The oracle keeps only a lookup table of $(x, H(x))$ to ensure deterministic responses. & nothing else.

9) All outputs are fixed length, typically l bits. (eg $l=256$)

$$H(x) \in \{0,1\}^l \quad \forall x \in \{0,1\}^*$$

LIMITATIONS

- No real hash function is a true random oracle
- Some schemes secure in ROM are insecure with real hash functions
- Programmability is not possible with actual hash functions